

Group 11 - Improving Robustness of Neural Code Search via Prompt-Evolved Query Reformulation

Haricharan Bharathi
North Carolina State University
USA
hbharat2@ncsu.edu

Sidharth Shambu
North Carolina State University
USA
sshambu@ncsu.edu

Abstract

Code search degrades badly when natural-language queries are rewritten to remove identifier overlap with the target code. We asked whether query rewriting could recover that performance without touching the retriever, and whether the answer depended on prompt quality or retriever architecture.

On the CodeXGLUE NL-code-search-Adv benchmark, BM25 drops from validation MRR 0.1654 on clean queries to 0.0518 on adversarial ones. Fixed-prompt rewriting improves clean retrieval but makes adversarial retrieval worse. Prompt evolution, following a PromptBreeder-inspired loop, never beats the seed prompt—every candidate scores lower.

GraphCodeBERT tells a different story: 0.5262 MRR on clean and 0.5285 on adversarial, essentially no degradation. But the same rewriting that marginally helps BM25 cuts GraphCodeBERT performance by 23–28%. The bottleneck is not prompt quality. It is the retriever. Lexical search cannot recover what identifier perturbation removes; semantic retrieval largely sidesteps the problem.

1 Introduction

Code search is a core software engineering activity. Developers routinely search for implementation patterns, library usage, and semantically related functions using natural-language descriptions. Neural code search has improved this process by learning aligned representations of code and language, with strong results reported for CodeBERT, GraphCodeBERT, CodeT5, and UniXcoder [9, 13, 14, 17]. Shared benchmarks such as CodeSearchNet and CodeXGLUE have accelerated progress by standardizing training and evaluation settings [8, 16].

Robustness is the weak spot. Semantics-preserving transformations—identifier renaming, dead-code insertion, style-preserving rewrites—can break model performance even when program behavior is unchanged [10–12, 22]. Code search is especially exposed here: many retrieval systems rely heavily on lexical overlap between the query and identifiers in the code.

This project asks whether robustness can be improved through query reformulation alone, without retraining the retriever. The starting hypothesis was that queries could be rewritten into forms that better match obfuscated code. The harder question turned out to be whether that approach is fundamentally limited, or whether its usefulness just depends on what retriever you pair it with.

To investigate this, we evaluate both a lexical retriever (BM25) and a semantic retriever (GraphCodeBERT) on the CodeXGLUE

adversarial code-search benchmark. BM25 provides a controlled deterministic baseline that exposes lexical dependence, while GraphCodeBERT provides a stronger semantic baseline through pre-trained code-language representations and data-flow-aware encoding.

BM25 collapses under adversarial identifier perturbation. Fixed-prompt rewriting recovers some clean-query performance through better token alignment but makes adversarial retrieval worse. Prompt evolution never beats the seed prompt. GraphCodeBERT, meanwhile, barely notices the perturbation—it outperforms BM25 by more than an order of magnitude on adversarial queries.

The main takeaway is not about prompts. Prompt quality is not the bottleneck. The retriever is. Query rewriting helps at the margins for lexical search, but semantic retrieval addresses the underlying problem directly.

The full implementation, experimental pipeline, and reproducibility scripts are available at: <https://github.com/shambu2k/ncs-prompt-query-reformulation>

1.1 Research Questions

- **RQ1:** How much does adversarial identifier perturbation degrade code-search performance on CodeXGLUE?
- **RQ2:** Can query rewriting improve retrieval quality under adversarial perturbations?
- **RQ3:** Does prompt evolution improve retrieval quality, and does its effectiveness depend on lexical retrieval (BM25) versus semantic retrieval (GraphCodeBERT)?

1.2 Contributions

- An expanded literature review covering twenty-eight papers across retrieval, pretrained models, robustness, and prompt optimization.
- A clearer methodological description with a dedicated implementation subsection for the BM25 baseline and the rewriting pipeline.
- An updated conceptual synthesis showing the limited overlap between robustness-aware query adaptation and mainstream code retrieval.
- A research design for evaluating prompt-evolved query reformulation on adversarial code search.
- Controlled comparison between lexical retrieval (BM25) and semantic retrieval (GraphCodeBERT) under fixed-prompt and prompt-evolved query rewriting.

2 Background and Motivation

Early code search systems were largely lexical or structural, relying on identifiers, APIs, documentation, and code structure to

retrieve relevant snippets [1–4]. Neural approaches later shifted the field toward representation learning by aligning natural-language queries and code in a shared embedding space [6]. Large pretrained code models further improved performance by learning from large corpora of code and natural language [9, 13, 14, 17].

Robustness has not kept pace. Multiple studies have shown that source-code models break under perturbations that preserve semantics but alter surface form [10–12, 22]. For code search, this suggests that performance is still partly driven by lexical alignment rather than deep semantic understanding.

Query reformulation is retriever-agnostic: it can be layered on top of any existing system without retraining. Earlier code-search work showed that query quality and alignment with code matter for retrieval [7, 20, 26]. Automatic prompt optimization can then improve the rewrite instructions themselves rather than handcrafting them [21, 25]. That combination—adversarial code search plus prompt-evolved query rewriting—is what this paper tests.

3 Related Work

3.1 Classical Code Search and Retrieval

Before neural code search, retrieval systems relied on lexical signals, structural metadata, and API information. Sourcerer indexed large repositories for practical code retrieval [1]. Hill et al. used phrasal representations from method signatures to improve search [2]. Portfolio retrieved functions along with their usage contexts [3]. CodeHow combined API knowledge with extended Boolean retrieval to rank results for natural-language queries [4]. DeepAPI treated API sequences as a semantic bridge between natural language and code [5]. Query representation mattered even before transformers.

3.2 Neural Code Search and Pretrained Code Models

DeepCS was an early system that embedded code and natural-language descriptions jointly [6]. CodeSearchNet followed with a large multi-language benchmark and standardized evaluation [8]. CodeBERT brought bimodal pretraining across programming and natural languages and set a strong baseline on code intelligence tasks [9]. GraphCodeBERT added data-flow information on top of token-level context [13]. CodeT5 used identifier-aware encoder-decoder pretraining [14], and UniXcoder extended this to a unified framework across understanding and generation [17]. CoSQA contributed real web queries to better reflect how developers actually search [15]. More recent work improved training through representation-level augmentation [18] and better negative-pair construction [19].

3.3 Benchmarks, Surveys, and Dataset Infrastructure

CodeXGLUE standardized a collection of program-understanding tasks and is the benchmark used here [16]. Recent surveys confirm that representation learning dominates the field: Di Grazia et al. covered lexical, semantic, and interaction-oriented retrieval [24]; Xie et al. focused on deep-learning methods and noted benchmark

dependence [27]; and Li et al. offered a three-dimensional taxonomy that shows fragmentation across retrieval paradigms and data sources [28]. Taken together, these surveys make clear that robust query adaptation for adversarial code search sits in a gap.

3.4 Query Reformulation and Retrieval Enhancement

Retrieval quality is sensitive to how queries are phrased, and this shows up throughout the code-search literature. CoaCor generated auxiliary code annotations and tuned retrieval via reinforcement learning [7]. Mao et al. proposed self-supervised query reformulation that avoids the need for labeled rewrite pairs [20]. Li et al. found that generated augmentations only help when they stay close to the target code distribution [26]. Outside software engineering, query rewriting for retrieval-augmented LLMs shows the same pattern: adapting the query can improve retrieval without touching the retriever [29].

3.5 Robustness and Adversarial Evaluation for Code Models

Code models fail under perturbations that preserve semantics. Yefet et al. showed that adversarial examples for code models can be built from simple source-level transformations [10]. Ramakrishnan et al. formalized the gap between program semantics and model behavior under perturbation [11]. Zhang et al. used Metropolis-Hastings-based identifier renaming to generate adversarial examples [12]. Chen et al. improved robustness through adversarial training and augmentation [22]. Follow-up work keeps finding the same vulnerability across tasks and model families [23, 32, 33]. Standard accuracy does not tell you much about retrieval quality when the lexical surface shifts.

3.6 Automatic Prompt Optimization

Prompt optimization offers a way to improve frozen language models without retraining. EvoPrompt treats prompt search as evolutionary optimization over discrete natural-language prompts [21]. PromptBreeder goes further through self-referential prompt evolution, letting the model improve its own mutation and scoring instructions [25]. Surveys of the area now recognize instruction search, exemplar selection, and discrete prompt evolution as a distinct subfield [30, 31]. None of this work has been applied to retrieval robustness in code search—that gap is what this paper addresses.

3.7 Synthesis and Research Gap

The survey covers four areas that do not talk to each other much: classical and neural code retrieval, pretrained code-language modeling, adversarial robustness, and prompt optimization. The first two are tightly linked through shared benchmarks. Robustness research intersects with pretrained models but has produced few retrieval-specific mitigations. Prompt optimization has grown up largely outside the code-search community. The connection that is missing—prompt-evolved query reformulation as a robustness tool for code search—is the focus of this paper.

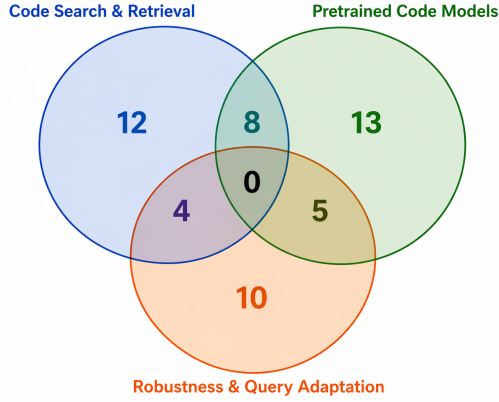


Figure 1: The largest overlap remains between code-search methods and pretrained code models, while robustness and query-adaptation work has limited direct intersection with mainstream retrieval.

4 Preliminary Literature Analysis

The survey covers twenty-eight papers, organized by research function: code search and retrieval, pretrained code models, and robustness with query adaptation.

Table 1 summarizes the broader literature coverage used in the revised review.

The distribution confirms the gap this paper addresses. Code retrieval and pretrained modeling are well covered. Robustness is increasingly studied, but usually as a model-side problem. Query reformulation and prompt optimization each appear in their own literatures, with almost no overlap in adversarial code search.

5 Methods

5.1 Dataset

Experiments use the CodeXGLUE NL-code-search-Adv benchmark for Python code search. Each example consists of a natural-language query paired with one correct code snippet among a fixed candidate set. The benchmark contains two retrieval conditions.

The clean condition uses the original developer-written docstring as the query. The adversarial condition replaces that docstring with a semantically equivalent but lexically dissimilar rewrite designed to reduce identifier overlap with the target code.

5.2 Retrieval Framework

The primary retriever is BM25, intentionally used as a deterministic lexical baseline rather than a neural semantic retriever. Candidate code snippets are indexed using the benchmark-provided pre-tokenized code tokens rather than re-tokenizing raw source code.

Standard Okapi BM25 is used with:

- $k_1 = 1.5$
- $b = 0.75$

No parameter tuning or model training is performed.

5.3 GraphCodeBERT Semantic Retrieval

BM25 is a useful lexical baseline but cannot be expected to handle semantic similarity when identifiers have been obfuscated. To test whether the BM25 limitations come from prompt quality or the retriever itself, we add a pretrained GraphCodeBERT-based dense retriever [13].

GraphCodeBERT is a strong pretrained model for code understanding that encodes data-flow information alongside token context. This makes it well-suited as a semantic baseline for adversarial code search, where lexical overlap between query and code is intentionally removed.

The GraphCodeBERT evaluation uses the pretrained microsoft/graphcodebert-base checkpoint without task-specific fine-tuning. Code snippets and natural-language queries are encoded into a shared embedding space, and retrieval is performed by ranking candidate code snippets according to cosine similarity with the query embedding. The dataset, benchmark splits, evaluation metric, and query-rewriting pipeline remain unchanged; only the retrieval mechanism is swapped.

Two questions motivate the GraphCodeBERT extension: does semantic retrieval reduce the clean-to-adversarial gap seen with BM25, and does prompt-based rewriting become more useful with a dense retriever? We used the same fixed-prompt rewriting and prompt-evolution setup from the BM25 experiments, keeping everything constant while swapping only the retriever.

5.4 Query Rewriting Baselines

Three retrieval conditions are evaluated:

- **Raw Query Retrieval:** original benchmark query directly used with BM25.
- **Fixed-Prompt Rewriting:** each query is rewritten once before retrieval using a fixed instruction template.
- **Prompt-Evolved Rewriting:** rewrite prompts are iteratively optimized using a PromptBreeder-inspired search loop.

The heuristic rewriter performs deterministic transformations including CamelCase splitting, underscore-to-space conversion, punctuation normalization, and whitespace cleanup.

The instruction-based rewriter uses an API-based backend with deterministic decoding, caching, retry logic, and concurrency control.

5.5 Prompt Evolution Framework

Prompt optimization follows a lightweight PromptBreeder-inspired hill-climbing strategy.

A frozen development subset of 300 adversarial validation queries is used for prompt search. For each candidate prompt:

- (1) rewrite development queries,
- (2) run BM25 retrieval,
- (3) compute MRR,
- (4) accept the candidate only if improvement exceeds ϵ .

Configuration:

- max iterations: 25
- patience: 5
- acceptance threshold: $\epsilon = 0.003$
- proposer temperature: 0.7

Table 1: Literature map used in the survey.

Category	Representative papers
Classical code retrieval	Sourcerer [1]; method-signature phrasal search [2]; Portfolio [3]; CodeHow [4]; DeepAPI [5]
Neural code search and benchmarks	DeepCS [6]; CodeSearchNet [8]; CodeXGLUE [16]; CoSQA [15]; representation-level augmentation [18]; negative-pair refinement [19]
Pretrained code models	CodeBERT [9]; GraphCodeBERT [13]; CodeT5 [14]; UniXcoder [17]
Query reformulation and retrieval adaptation	CoaCor [7]; SSQR [20]; generation-augmented retrieval analysis [26]; query rewriting for retrieval-augmented LLMs [29]
Robustness and adversarial code modeling	Adversarial Examples for Models of Code [10]; Semantic Robustness of Models of Source Code [11]; MHM attack [12]; robustness of code-pretrained models [22]; broader adversarial attack studies [23, 32, 33]
Surveys and synthesis	Code search survey [24]; deep-learning-based code-search survey [27]; three-dimensional source-code-search taxonomy [28]; automatic prompt optimization surveys [30, 31]
Prompt optimization	EvoPrompt [21]; PromptBreeder [25]

Table 2: Dataset statistics used in experiments.

Split	Condition	Records
valid	clean	9,604
valid	adversarial	9,604
test	clean	19,210
test	adversarial	19,210

- scoring temperature: 0.0

5.6 Metrics

The primary evaluation metric is Mean Reciprocal Rank (MRR):

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i}$$

Robustness degradation is measured as:

$$\text{Robustness Drop} = MRR_{\text{clean}} - MRR_{\text{adv}}$$

6 Experimental Results

6.1 BM25 Baseline

Table 3: Raw BM25 baseline.

Split	Clean MRR	Adv MRR
Validation	0.1654	0.0518
Test	0.1338	0.0421

Adversarial performance is more than three times lower than clean performance, confirming that lexical overlap is a major dependency for BM25 retrieval.

6.2 Fixed-Prompt Rewriting

Heuristic rewriting improves clean-query retrieval by improving token alignment, but worsens adversarial retrieval because surface normalization cannot recover missing semantic links.

6.3 Prompt Evolution Results

All generated prompt mutations performed worse than the original seed prompt. Prompt evolution therefore converged immediately to the seed prompt.

6.4 GraphCodeBERT Semantic Retrieval Results

Raw GraphCodeBERT substantially outperforms BM25 across both clean and adversarial settings, with more than a tenfold improvement on adversarial retrieval. Unlike BM25, GraphCodeBERT shows almost no robustness degradation under identifier perturbation. Semantic retrieval largely sidesteps the lexical failure mode rather than trying to correct for it.

The rewriting result is counterintuitive: the same strategy that slightly helps BM25 harms GraphCodeBERT by 23–28%. Heuristic rewriting strips out lexical cues that dense retrieval models already use, trading semantic precision for surface normalization.

6.5 Qualitative Examples

We manually reviewed examples from the adversarial test split to see where rewriting helped and where it failed.

For long-form queries like “Copy a file from source to target,” identifier splitting and punctuation normalization exposed lexical matches in the indexed code and documentation, moving the correct result to rank 1 from no match. Functionality-focused queries like this tended to benefit from rewriting.

For identifier-sensitive queries like “Enable %autopx mode by saving the original run_cell and installing pxrun_cell,” rewriting destroyed the signal. Exact tokens like run_cell and pxrun_cell became “run cell” and “pxrun cell,” and the target code fell out of top results entirely. Identifier-heavy queries where the exact string matters are where rewriting consistently failed.

The pattern matches the quantitative results: rewriting helps when it improves token overlap without losing meaningful identifiers, and hurts when it trades away the only lexical anchors connecting query to code.

7 Discussion

The bottleneck in adversarial code search is not prompt quality. It is the retriever.

Table 4: Fixed-prompt heuristic rewriting results.

Method	Clean MRR	Adv MRR	Robustness Drop	Relative Drop
Raw Query BM25 (valid)	0.1654	0.0518	0.1136	68.7%
Fixed Prompt Rewrite (valid)	0.1878	0.0474	0.1404	74.8%
Raw Query BM25 (test)	0.1338	0.0421	0.0917	68.5%
Fixed Prompt Rewrite (test)	0.1537	0.0382	0.1155	75.1%

Table 5: Prompt evolution results on development subset.

Iter	Candidate	Seed	Delta	Accepted
Seed	0.0556	0.0556	–	Yes
1	0.0352	0.0556	-0.0204	No
2	0.0357	0.0556	-0.0199	No
3	0.0389	0.0556	-0.0166	No
4	0.0366	0.0556	-0.0190	No
5	0.0421	0.0556	-0.0135	No

Table 6: Validation results comparing BM25 and GraphCodeBERT.

Method	Clean MRR	Adv MRR	Robustness Drop	Relative Drop
Raw BM25	0.1654	0.0518	0.1136	68.7%
Fixed Rewrite BM25	0.1878	0.0474	0.1404	74.8%
Raw GraphCodeBERT	0.5262	0.5285	-0.0023	–
Fixed Rewrite GraphCodeBERT	0.4050	0.3937	0.0113	2.8%

Table 7: Test results comparing BM25 and GraphCodeBERT.

Method	Clean MRR	Adv MRR	Robustness Drop	Relative Drop
Raw BM25	0.1338	0.0421	0.0917	68.5%
Fixed Rewrite BM25	0.1537	0.0382	0.1155	75.1%
Raw GraphCodeBERT	0.5017	0.4966	0.0051	1.0%
Fixed Rewrite GraphCodeBERT	0.3749	0.3556	0.0193	5.1%

BM25 works when lexical overlap connects query to code. Clean queries benefit from identifier splitting and surface normalization because rewritten tokens land closer to indexed vocabulary. Adversarial queries are built to remove that overlap. Once it is gone, no amount of surface normalization brings it back.

Fixed-prompt rewriting improves clean retrieval because it optimizes token form. It worsens adversarial retrieval for the same reason. Prompt evolution confirms this: if prompt quality were the limiting factor, iterative search would find better prompts. Instead, every mutation underperforms the seed, and the evolved rewrite offers nothing over the fixed baseline.

GraphCodeBERT tells a different story. It achieves near-zero robustness degradation and outperforms raw BM25 by more than ten times on adversarial queries. Semantic retrieval sidesteps the lexical mismatch problem rather than trying to patch it.

The rewriting result with GraphCodeBERT is worth pausing on: the same pipeline that marginally helps BM25 cuts GraphCodeBERT performance by 23–28%. The rewriter was designed to improve lexical alignment, and it does—at the cost of semantic and structural information that dense retrievers already use. Rewriting is not

universally helpful. Its value depends entirely on what the retriever is sensitive to.

The practical implication is that prompt optimization is often working on the wrong problem. Query rewriting can help at the margins for lexical retrieval, but it cannot compensate for a system that matches words rather than meaning. The more productive path is better retrieval architecture—dense encoders and cross-encoder reranking—not more sophisticated prompts.

8 Conclusion

We asked whether prompt-evolved query reformulation could improve adversarial code search without changing the retriever. The short answer is no—not in any meaningful way.

BM25 degrades severely when identifier overlap is removed. Fixed-prompt rewriting helps on clean queries by improving token alignment, but makes adversarial queries worse. Prompt evolution never beats the seed. None of these prompt-side interventions compensate for what lexical retrieval fundamentally cannot do.

GraphCodeBERT, by contrast, loses almost nothing to adversarial perturbation. It outperforms BM25 on adversarial queries by more than an order of magnitude, not because of better prompts but because it does not depend on lexical overlap in the first place.

If you are trying to build a robust code search system, the retriever matters more than the prompt. Query rewriting is a useful normalization trick, but it is not a robustness strategy. Semantic retrieval is.

References

- [1] Suman Bajracharya, Trung Ngo, Erik Linstead, Yannis Dou, Paul Rigor, Pierre Baldi, and Cristina Lopes. 2006. Sourcerer: An Internet-scale software repository. In *Proceedings of the ICSE Workshop on Mining Software Repositories*.
- [2] Emily Hill, Lori Pollock, and K. Vijay-Shanker. 2011. Improving source code search with natural language phrasal representations of method signatures. In *Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering*.
- [3] Collin McMillan, Mark Grechanik, Denys Poshyvanyk, Qing Xie, and Chen Fu. 2011. Portfolio: Finding relevant functions and their usage. In *Proceedings of the 33rd International Conference on Software Engineering*.
- [4] Fei Lv, Hongyu Zhang, Jian-Guang Lou, Shaowei Wang, Dongmei Zhang, and Jianjun Zhao. 2015. CodeHow: Effective code search based on API understanding and extended Boolean model. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering*.
- [5] Xiaodong Gu, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. 2016. Deep API learning. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*.
- [6] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*.
- [7] Ziyu Yao, Daniel S. Weld, Wei-Peng Chen, and Huan Sun. 2019. CoaCor: Code annotation for code retrieval with reinforcement learning. In *Proceedings of the World Wide Web Conference*.
- [8] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436.

- [9] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of EMNLP*.
- [10] Noam Yefet, Uri Alon, and Eran Yahav. 2020. Adversarial examples for models of code. *Proceedings of the ACM on Programming Languages* 4(OOPSLA).
- [11] Goutham Ramakrishnan, Jordan Henkel, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas Reps. 2020. Semantic robustness of models of source code. arXiv:2002.03043.
- [12] Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, and Zhi Jin. 2020. Generating adversarial examples for holding robustness of source code processing models. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [13] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations*.
- [14] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of EMNLP*.
- [15] Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. 2021. CoSQA: 20,000+ web queries for code search and question answering. In *Proceedings of ACL*.
- [16] Shangqing Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of NeurIPS Datasets and Benchmarks*.
- [17] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of ACL*.
- [18] Hongjin Li, Muhao Chen, and Tianyi Zhang. 2022. Exploring representation-level augmentation for code search. In *Proceedings of EMNLP*.
- [19] Hongjin Li, Muhao Chen, and Tianyi Zhang. 2023. Rethinking negative pairs in code search. In *Proceedings of EMNLP*.
- [20] Yuetian Mao, Chengcheng Wan, Yuze Jiang, and Xiaodong Gu. 2023. Self-supervised query reformulation for code search. arXiv:2307.00267.
- [21] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2023. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. arXiv:2309.08532.
- [22] Nuo Chen, Qiushi Sun, Xiang Li, Ming Gao, and Lingjuan Lyu. 2023. Evaluating and enhancing the robustness of code pre-trained models. In *Findings of EMNLP*.
- [23] Wenxuan Sun, Ge Li, Xia Li, and Zhi Jin. 2023. An extensive study on adversarial attack against pre-trained models of code. arXiv:2311.07553.
- [24] Lucia Di Grazia, Nargis Fatema, and Vladimir Filkov. 2023. Code search: A survey of techniques for finding code. *ACM Computing Surveys*.
- [25] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Self-referential self-improvement via prompt evolution. In *Proceedings of ICML*.
- [26] Hongjin Li, Muhao Chen, and Tianyi Zhang. 2024. Rewriting the code: A simple method for large language model augmented code search. In *Proceedings of ACL*.
- [27] Jiale Xie, Yutong Xie, Yaqin Zhou, and colleagues. 2024. Survey of code search based on deep learning. *ACM Transactions on Software Engineering and Methodology*.
- [28] Xia Li, Ge Li, and Zhi Jin. 2024. A survey of source code search: A three-dimensional taxonomy and open challenges. *ACM Transactions on Software Engineering and Methodology*.
- [29] Xinyu Ma, Yizhu Jiao, Yixin Cao, and colleagues. 2023. Query rewriting in retrieval-augmented large language models. arXiv:2305.14283.
- [30] Yongchao Zhou, Yutong Wu, and colleagues. 2025. A systematic survey of automatic prompt optimization. arXiv:2502.16923.
- [31] Yunfan Liu, Hanze Dong, and colleagues. 2025. A survey of automatic prompt engineering. arXiv:2502.11560.
- [32] Kai Wang, and colleagues. 2025. Cross-model deception: Transferable adversarial attacks on code language models for code search. OpenReview preprint.
- [33] Anonymous. 2026. CLARC: C/C++ benchmark for robust code search. OpenReview preprint.